

Neural networks - Project 2

Class imbalance problem

Lenka Husárová, Martin Mislovič, Hana Magyarová

Last updated: April 16, 2025

✓ Task

Our task was image classification of animal species and the impact of imbalanced datasets.

PAPER: <https://arxiv.org/pdf/1710.05381>

✓ Metrics

The project explores various techniques to address the imbalance (data augmentation, resampling strategies (oversampling and undersampling)).

We agreed to use following metrics to understand how imbalance affects correct classification:

- accuracy
- precision
- F1-score
- recall
- top1
- top5

We also used confusion matrices for evaluation.

✓ Code

- **EDA.ipynb** - This notebook contains our initial data exploration and preprocessing steps.

Key activities include:

- analyzing the distribution of images across different animal classes
- examining image resolutions and size variance
- identifying class imbalance as a major challenge
- detecting mislabeled images and inconsistencies in the dataset
- preparing the dataset for modeling by applying preprocessing steps

- **model.ipynb** - This notebook focuses on building, training, and evaluating classification models.

It includes:

- implementation of multiple architectures for image classification
 - experimentation with different training strategies
 - testing various techniques to handle class imbalance
- **translate.py** - Python dict containing mapping between italian (original) and english names of animals (since the dataset was initially posted with italian names)

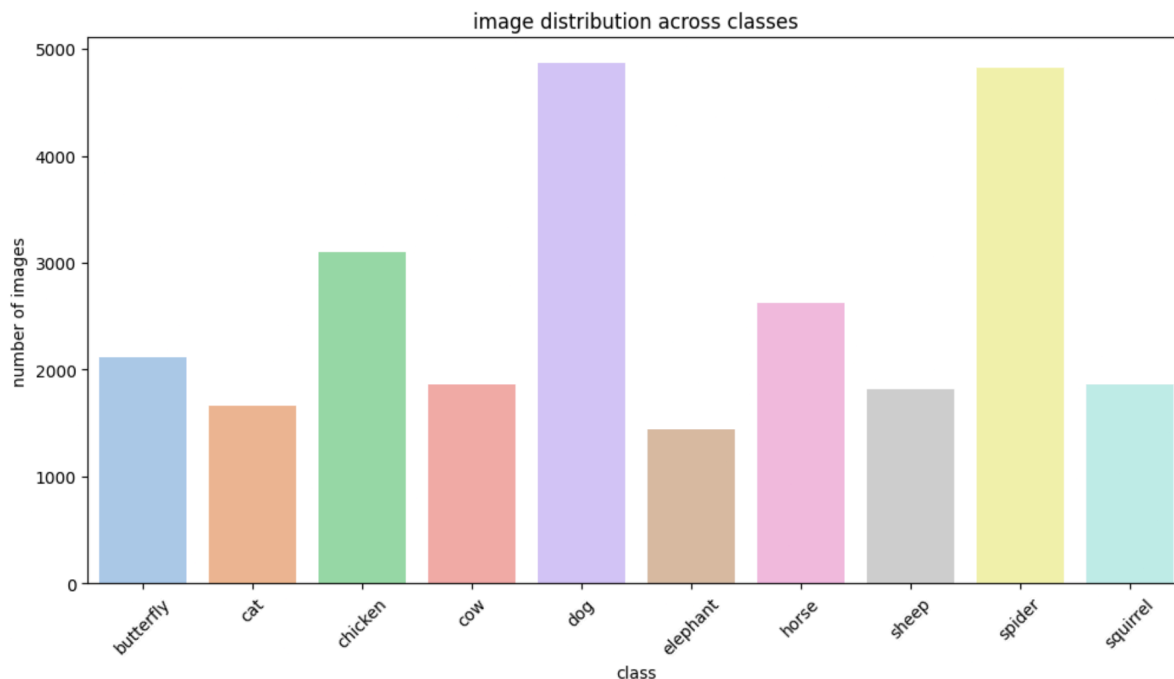
✓ Dataset

[Available here.](#)

It contains about 28K medium quality animal images belonging to 10 categories: dog, cat, horse, spider, butterfly, chicken, sheep, cow, squirrel, elephant.

✓ EDA

In the EDA notebook, we performed an overall statistical analysis of the dataset to better understand its structure and quality. We started by plotting a histogram to visualize the distribution of images across different classes, which immediately revealed a **significant class imbalance** (e.g., some classes contain as few as 1.5k images, while others have over 5k). We then displayed a sample of images from each class to assess image quality and labeling accuracy. During this process, we identified several **mislabeled or incorrectly categorized images**. Additionally, we analyzed image resolutions and found a **high variance in image sizes**, which may affect model performance and will be addressed during preprocessing.



Histogram of image distribution across classes

✓ Preprocessing

In the preprocessing stage, we implemented a custom **DataModule** class to handle data loading, augmentation, and balancing. All images were **resized to 224×224** pixels for consistency, and training images underwent

a **series of augmentations**—including random horizontal flips, rotations, affine transformations, and color jittering—to improve model generalization and simulate real-world variability. All datasets (train, validation, test) were **normalized** using the dataset-specific **mean** and **standard deviation** values.

To address class imbalance, we incorporated optional "online" oversampling or undersampling modes during training data preparation via a BalancerWrapper. The dataset was split (**70:15:15 ratio**) into training (70%), validation (15%), and test (15%) subsets using a fixed random seed to ensure reproducibility. Validation and test sets were only resized and center-cropped to 224×224, without augmentation, to maintain evaluation consistency and to avoid data leakage.

✓ Models and training

All models accept RGB images (3 channels) and produce predictions for 10 classes.

✓ AlexNet

It is based on the classic AlexNet architecture with some modifications. The model consists of a deep stack of convolutional layers followed by fully connected layers. It integrates **batch normalization** after each convolution and linear layer to stabilize training and improve convergence. **Dropout** is applied in the fully connected layers to **reduce overfitting**. The model ends with a **softmax** activation to output class probabilities.

```
Input (3 x 224 x 224)
|
├─ Conv2d(3 → 96, 11x11, stride=4)
├─ BatchNorm2d(96)
├─ ReLU
├─ MaxPool2d(3x3, stride=2)
|
├─ Conv2d(96 → 256, 5x5, padding=2)
```

```

└─ BatchNorm2d(256)
└─ ReLU
└─ MaxPool2d(3x3, stride=2)
|
└─ Conv2d(256 → 384, 3x3, padding=1)
└─ BatchNorm2d(384)
└─ ReLU
|
└─ Conv2d(384 → 384, 3x3, padding=1)
└─ BatchNorm2d(384)
└─ ReLU
|
└─ Conv2d(384 → 256, 3x3, padding=1)
└─ BatchNorm2d(256)
└─ ReLU
└─ MaxPool2d(3x3, stride=2)
|
└─ Flatten
|
└─ Linear(256*5*5 → 4096)
└─ BatchNorm1d(4096)
└─ ReLU
└─ Dropout(0.5)
|
└─ Linear(4096 → 4096)
└─ BatchNorm1d(4096)
└─ ReLU
└─ Dropout(0.5)
|
└─ Linear(4096 → 10)
└─ Softmax

```

✓ SimpleNet

SimpleNet is a custom convolutional neural network designed for multi-class image classification. It is using **batch normalization** and **ReLU** activations. The model includes **max-pooling** layers to reduce spatial dimensions and a compact fully connected section that maps the learned features to 10 output classes. **Dropout** is used in the fully connected

layers to prevent overfitting, and the final **softmax** layer provides class probabilities.

```
Input (3 x 224 x 224)
|
├─ Conv2d(3 → 96, 11x11, stride=4)
├─ BatchNorm2d(96)
├─ ReLU
├─ MaxPool2d(3x3, stride=2)
|
├─ Conv2d(96 → 256, 5x5, padding=2)
├─ BatchNorm2d(256)
├─ ReLU
├─ MaxPool2d(3x3, stride=2)
|
├─ Conv2d(256 → 384, 3x3, padding=1)
├─ BatchNorm2d(384)
├─ ReLU
|
├─ Conv2d(384 → 384, 3x3, padding=1)
├─ BatchNorm2d(384)
├─ ReLU
|
├─ Conv2d(384 → 256, 3x3, padding=1)
├─ BatchNorm2d(256)
├─ ReLU
├─ MaxPool2d(3x3, stride=2)
|
├─ Flatten
|
├─ Linear(256*5*5 → 2048)
├─ BatchNorm1d(2048)
├─ ReLU
├─ Dropout(0.5)
|
├─ Linear(2048 → 512)
├─ BatchNorm1d(512)
├─ ReLU
├─ Dropout(0.5)
```

```

|
├─ Linear(512 → 10)
└─ Softmax

```

✓ SimpleNetV3

This model represents the final refinement of SimpleNet. Our initial SimpleNet performed well on undersampled dataset but struggled to extract more information from the oversampled set. This new version addresses these concerns with slightly modified convolution layer size - **smaller kernels** at the start and more channels at the end. Furthermore this version uses **more pooling layers** as well as **global average pooling** to give the classification head as much information as possible.

Additionally, standard ReLU activations were replaced with **LeakyReLU** to further increase performance.

The classification head was further **simplified to reduce overfitting**. This version also incorporates dropout between the convolutional layers to reduce the overfitting which was a significant issue with the initial architecture.

```

Input (3 x 224 x 224)
|
├─ Conv2d(3 → 64, 5x5, stride=2, padding=2)
├─ BatchNorm2d(64)
├─ LeakyReLU(0.1)
├─ Dropout2d(0.1)
├─ MaxPool2d(3x3, stride=2)
|
├─ Conv2d(64 → 128, 3x3, stride=1, padding=1)
├─ BatchNorm2d(128)
├─ LeakyReLU(0.1)
├─ Dropout2d(0.2)
└─ MaxPool2d(3x3, stride=2)

```

```

|
├─ Conv2d(128 → 256, 3x3, stride=1, padding=1)
├─ BatchNorm2d(256)
├─ LeakyReLU(0.1)
├─ Dropout2d(0.3)
|
├─ Conv2d(256 → 256, 3x3, stride=1, padding=1)
├─ BatchNorm2d(256)
├─ LeakyReLU(0.1)
├─ MaxPool2d(3x3, stride=2)
|
├─ Conv2d(256 → 512, 3x3, padding=1)
├─ BatchNorm2d(512)
├─ LeakyReLU(0.1)
├─ AdaptiveAvgPool2d((1, 1))
|
├─ Flatten
|
├─ Linear(512 → 256)
├─ BatchNorm1d(256)
├─ LeakyReLU(0.1)
├─ Dropout(0.5)
├─ Linear(256 → 10)
└─ Output (logits)

```

✓ Experiments

For the classification task, we experimented with several CNN architectures to evaluate their performance on our dataset. Our main models include AlexNet, SimpleNet and SimpleNetV3, which we customized and trained extensively. These models offered a good balance between performance and training feasibility.

We looked at other models, including LeNet and ResNet, but decided not to use them in our final experiments. LeNet did not perform well because it is too simple. Although ResNet is powerful, it takes too long to train, which didn't fit our computational limits. The models we chose still require a lot of training time, but it is manageable (with full runs taking about 8 hours to complete with our resources).

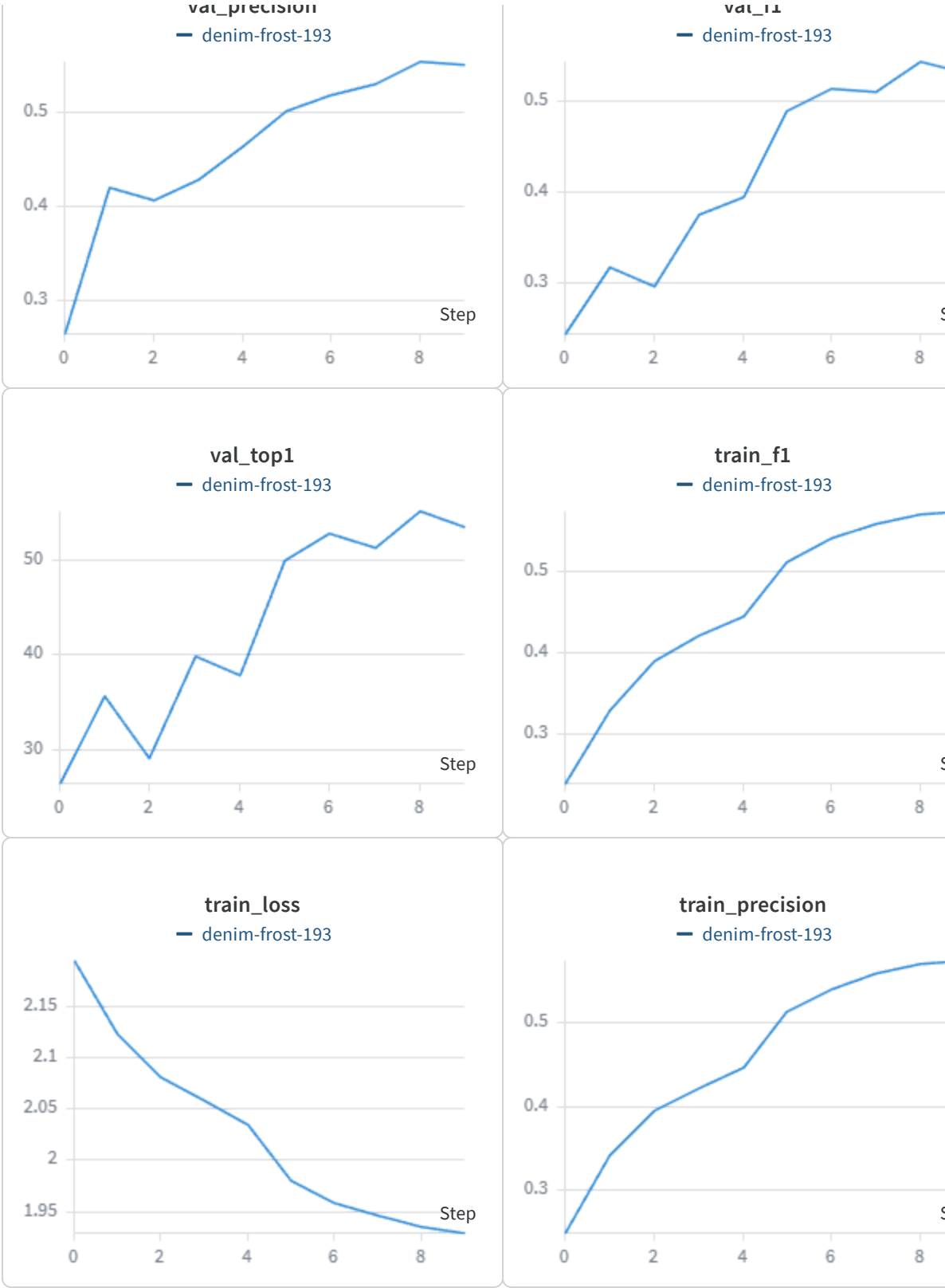
✓ Test run of AlexNet

We started our experimentation with basic AlexNet model.

As an initial experiment, we trained our first model on an undersampled version of the dataset to address class imbalance. The model was trained for 10 epochs using a learning rate of 0.0001, weight decay of 0.1, and a batch size of 64. After training, the model achieved approximately 58% accuracy, precision, recall, and F1-score on the training set.

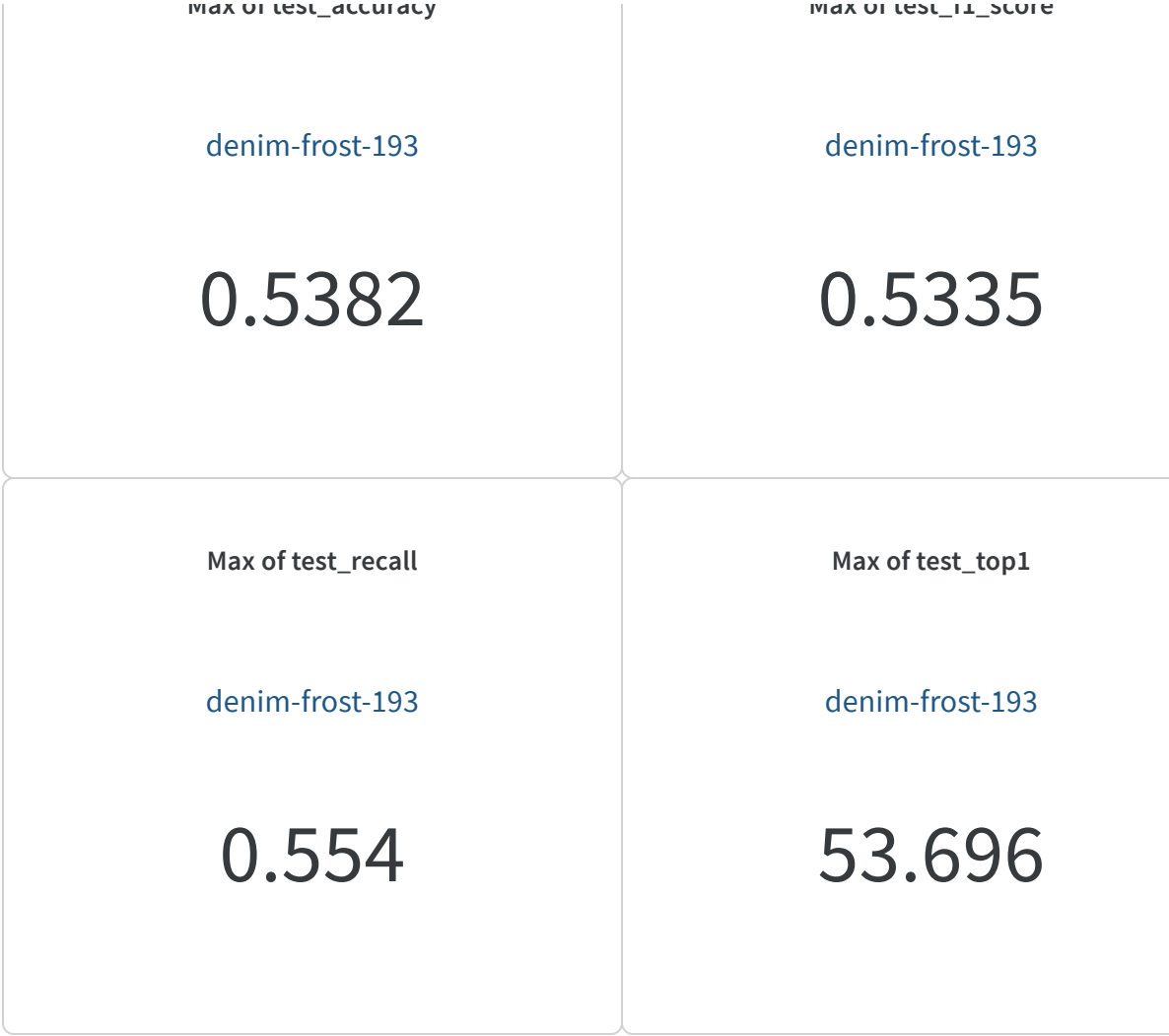
On the validation set, the model reached a top-1 accuracy of around 55% and a top-5 accuracy of approximately 92%. In terms of validation metrics, it achieved a precision of ~60%, recall of 55%, and an F1-score of 51%. These results provided a baseline for further tuning and comparison with other models.





Max of test accuracy

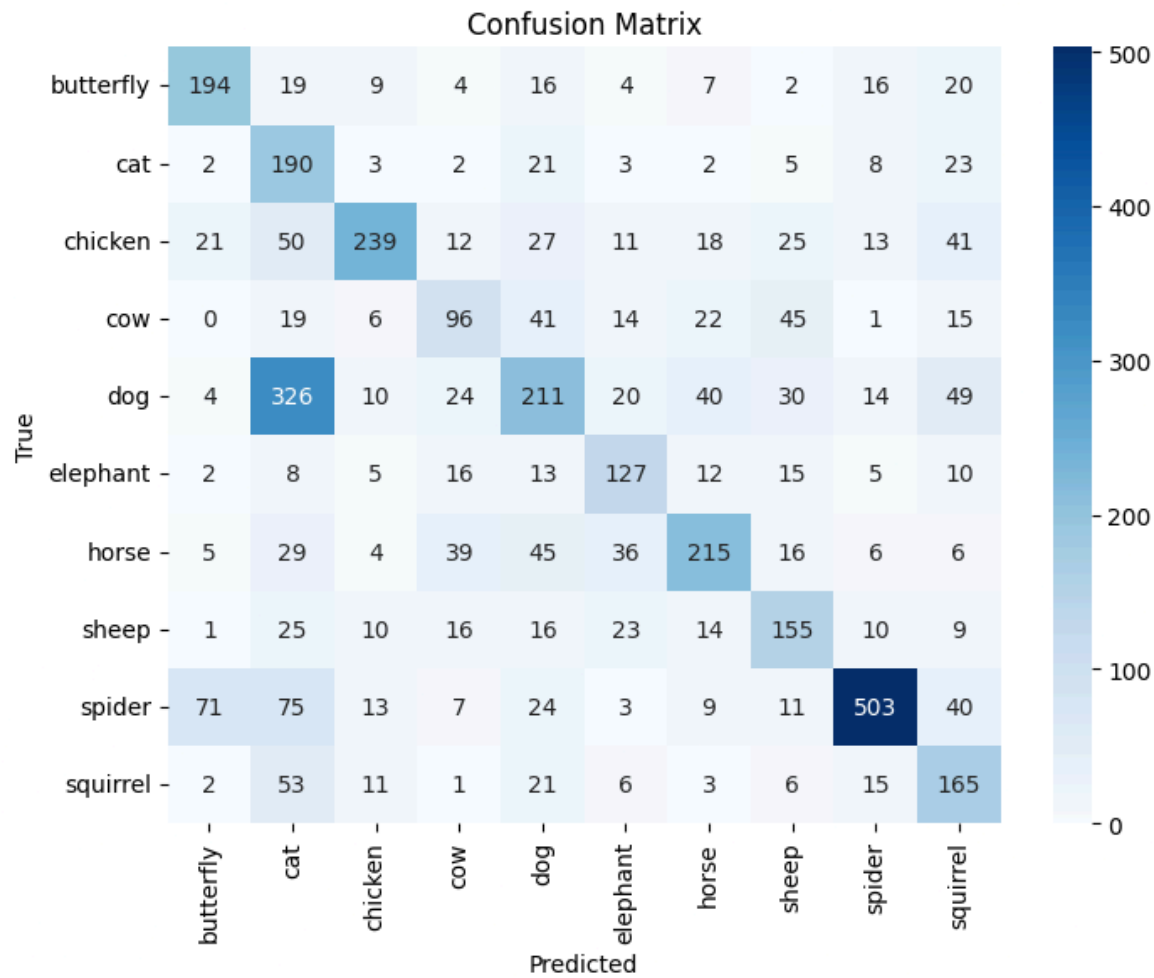
Max of test f1 score



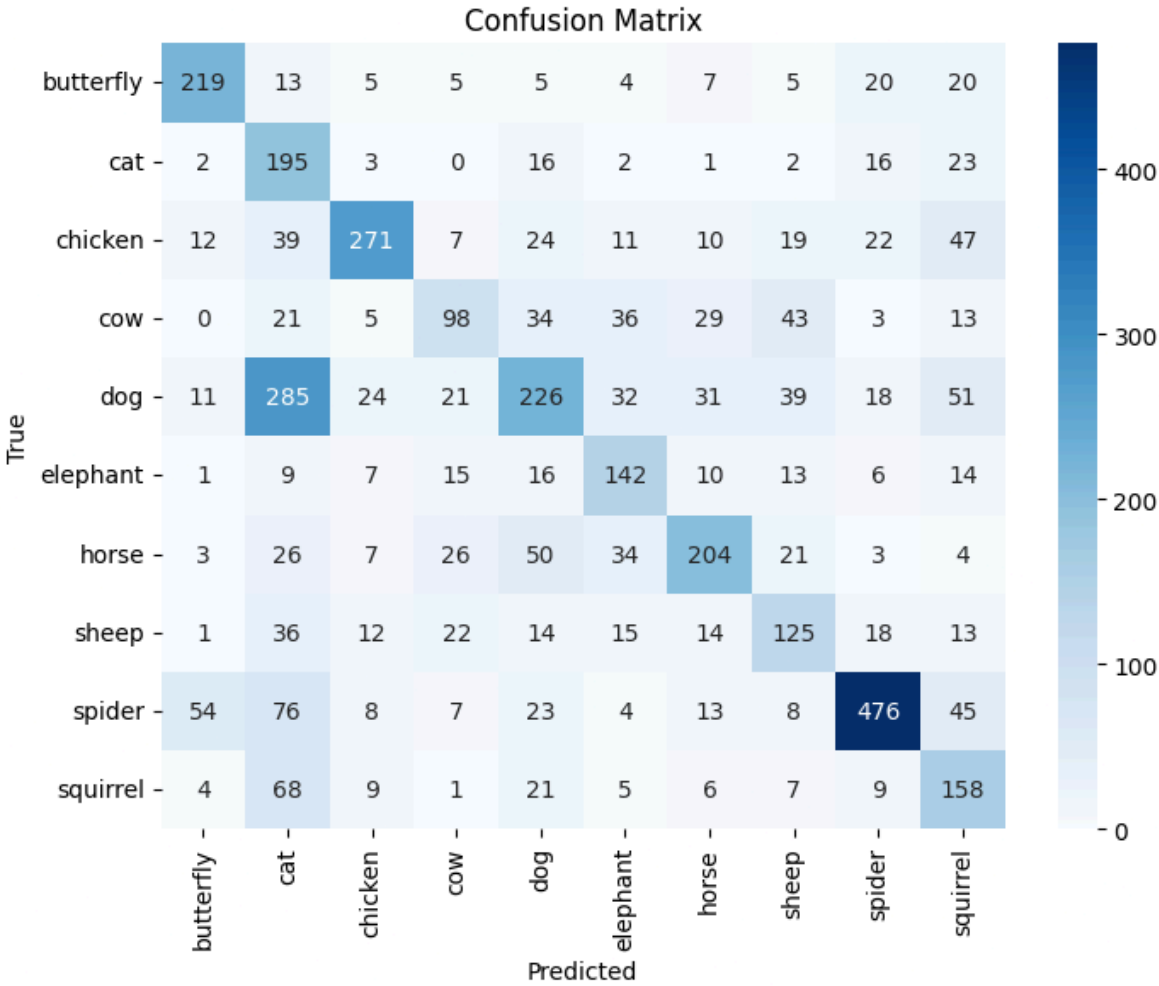
Dataset	Metric	Value
Train	Accuracy	~58%
	Precision	~58%
	Recall	~58%
	F1-Score	~58%

Validation

Test	Accuracy	~53%
	Top-1 Accuracy	~53%
	Top-5 Accuracy	~92%
	Precision	~55%
	Recall	~55%
	F1-Score	~53%
	Accuracy	~54%
	Precision	~55%
	Recall	~55%
	F1-Score	~53%
	Top-1 Accuracy	~53%
	Top-5 Accuracy	~92%



Confusion matrix for the validation set

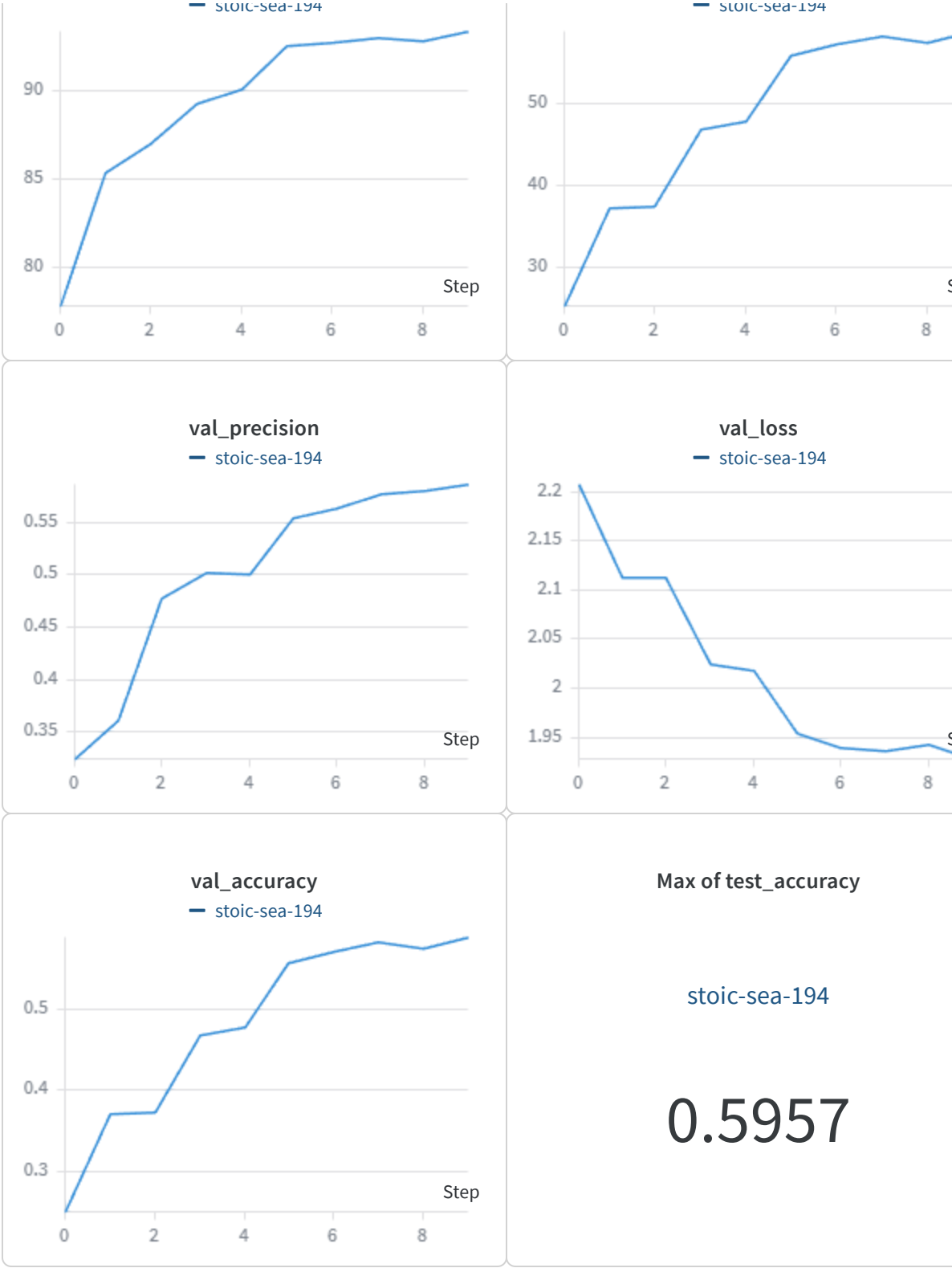


Confusion matrix for the test set

✓ Test run of SimpleNet

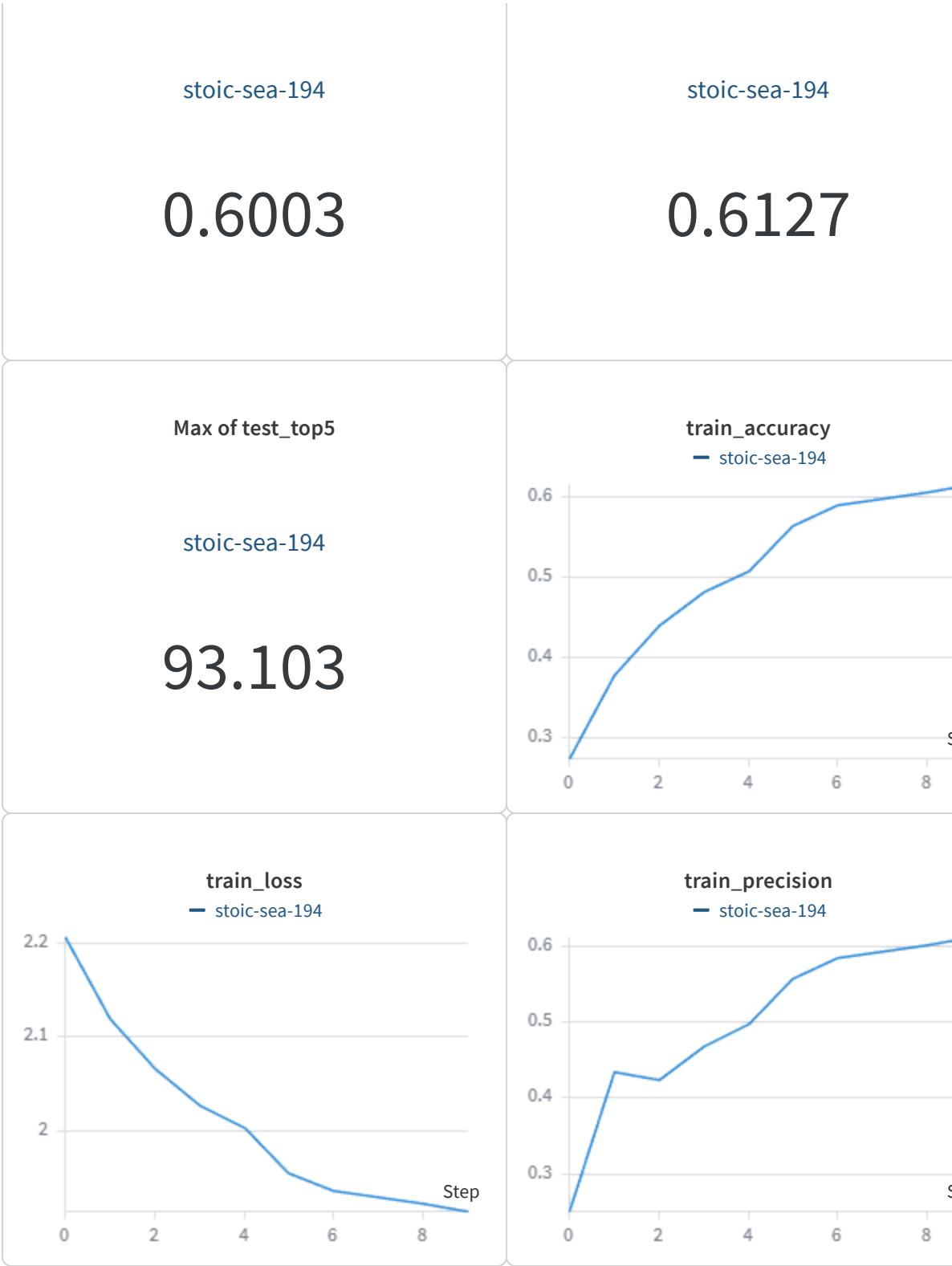
We trained SimpleNet with exactly the same conditions as the previous model.



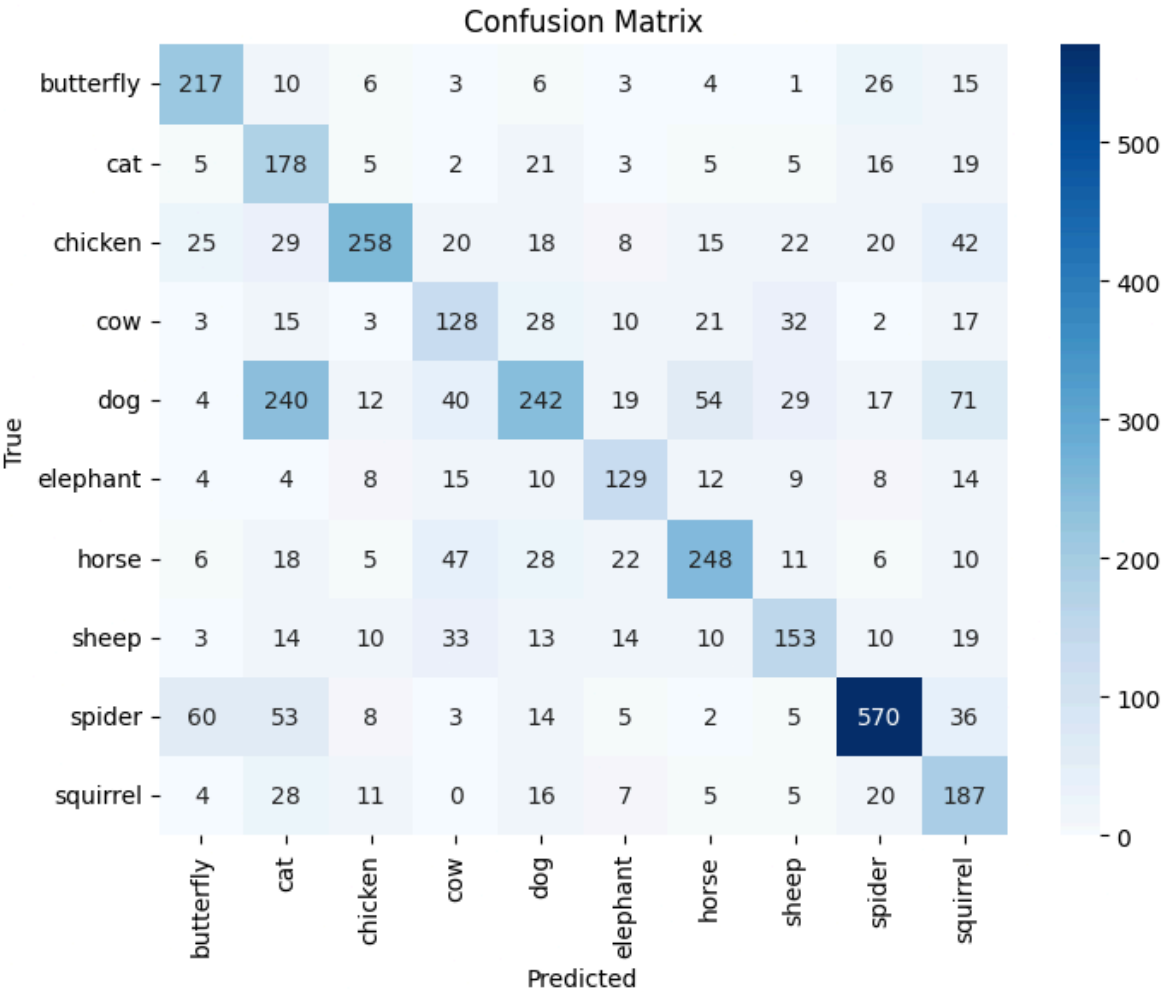


Max of test_precision

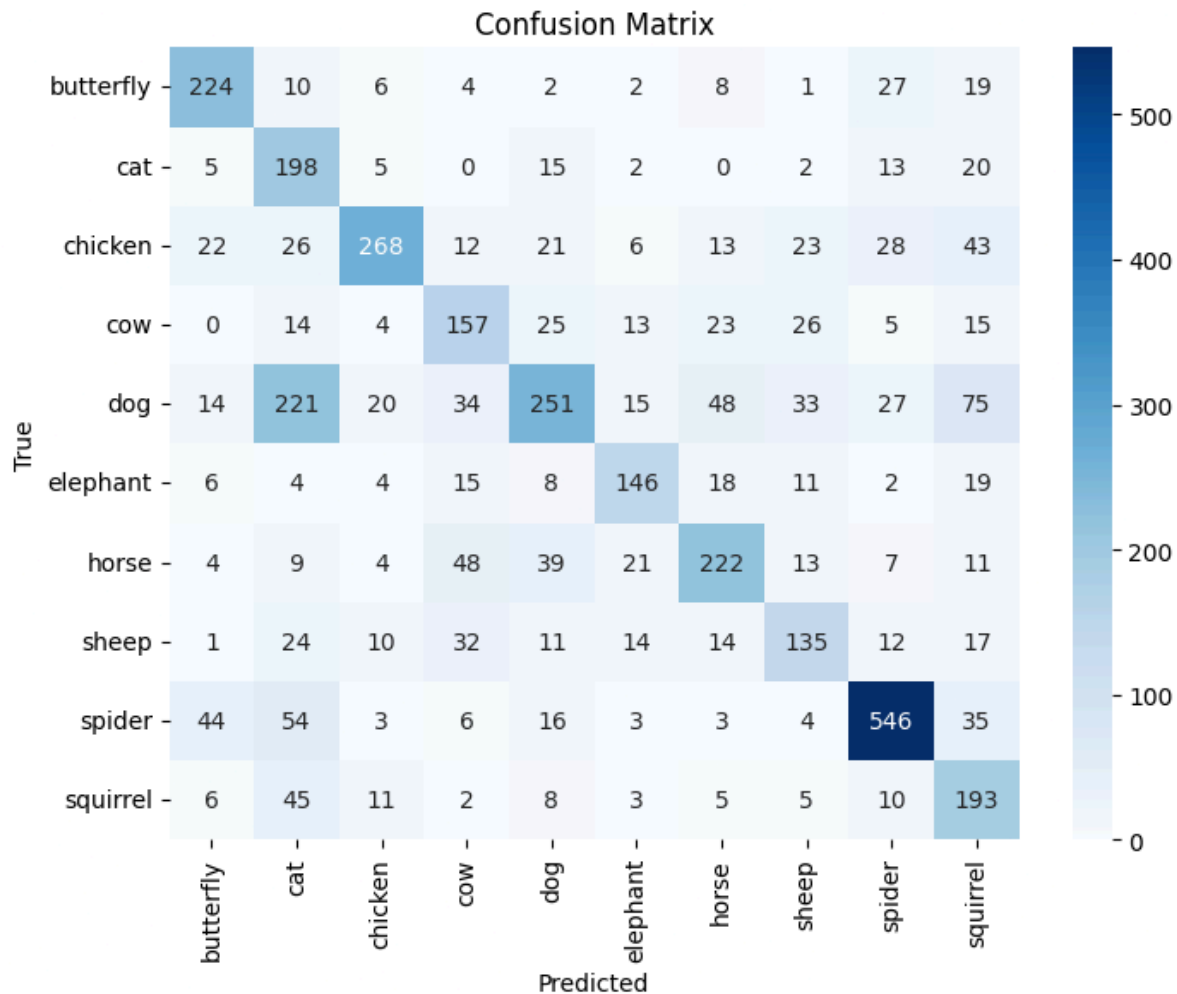
Max of test_recall



Dataset	Metric	Value
Train	Accuracy	~61%
	Precision	~61%
	Recall	~61%
	F1-Score	~61%
Validation	Top-1 Accuracy	~58%
	Top-5 Accuracy	~93%
	Accuracy	~58%
	Precision	~58%
	Recall	~60%
	F1-Score	~57%
Test	Accuracy	~60%
	Precision	~60%
	Recall	~61%
	F1-Score	~58%
	Top-1 Accuracy	~59%
	Top-5 Accuracy	~93%



Confusion matrix for the validation set



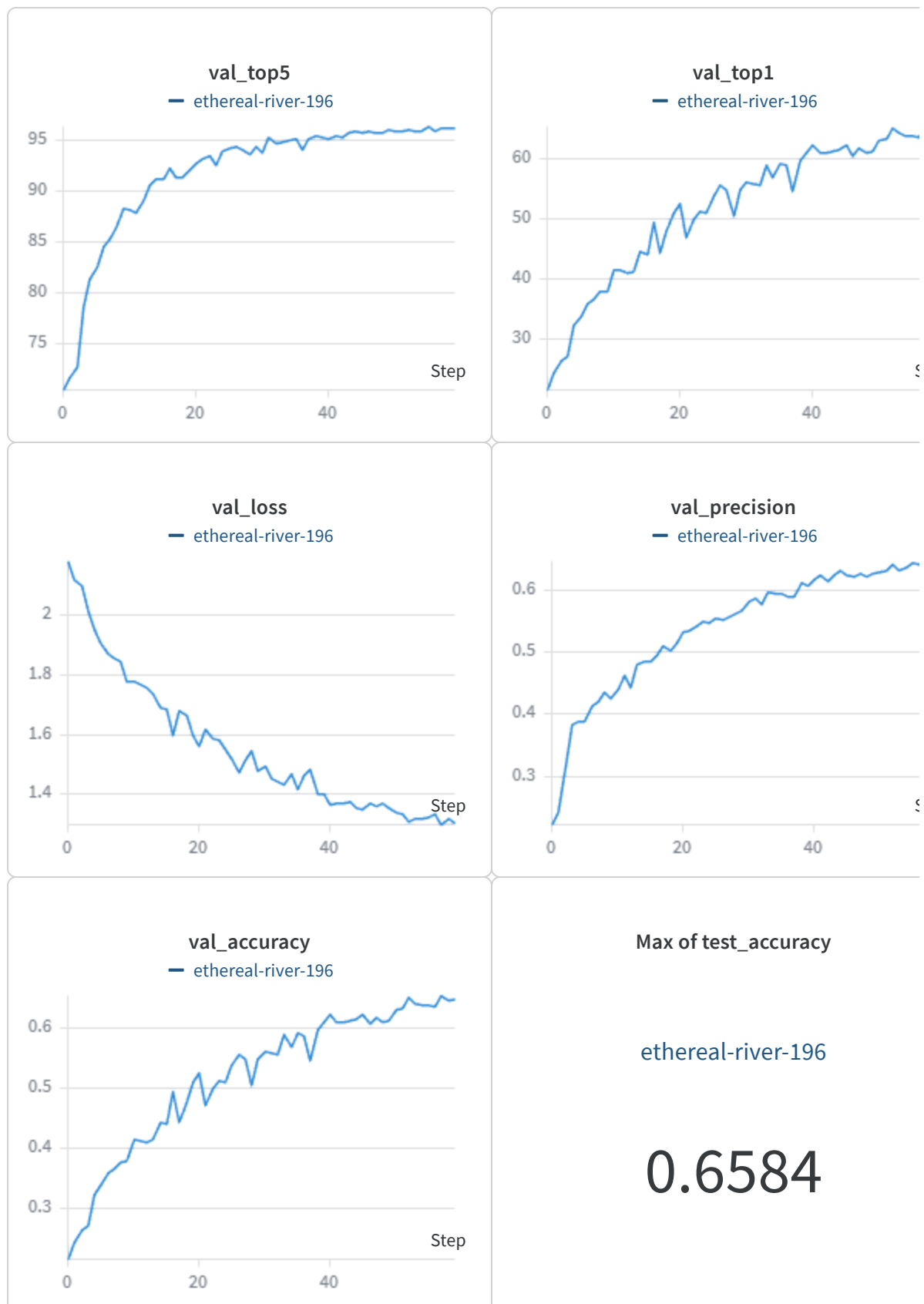
Confusion matrix for the test set

- ✓ Final runs with SimpleNetV3
- ✓ Undersampled dataset

We trained the SimpleNetV3 model on an undersampled dataset to address class imbalance. Each class had exactly 1000 samples. The model was optimized using the **AdamW optimizer** with a **learning rate** of 0.0001 and a **weight decay** of 0.1 to promote generalization. To improve convergence, we used a **Cosine Annealing learning rate scheduler** with a minimum learning rate of 1e-5 over **60 epochs**.

Training was performed with a **batch size** of 64, and the loss was calculated using **CrossEntropyLoss** with label smoothing (0.1) to reduce overconfidence in predictions. This setup provided a balanced and regularized learning process suitable for handling our high-variance, previously imbalanced dataset.

SimpleNetV3 demonstrated solid generalization performance. The model achieved ~63% accuracy on both the training and validation sets.



Max of test_top5

ethereal-river-196

96.052

Max of test_recall

ethereal-river-196

0.6774

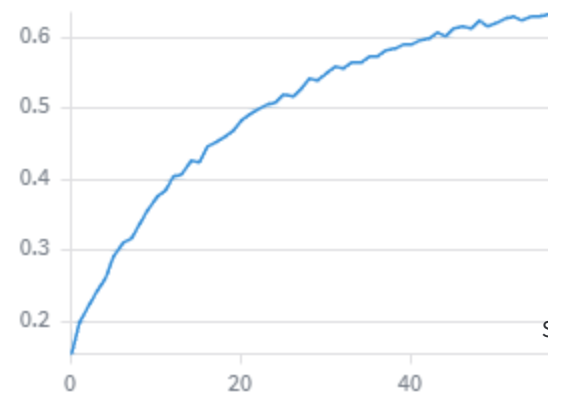
Max of test_precision

ethereal-river-196

0.656

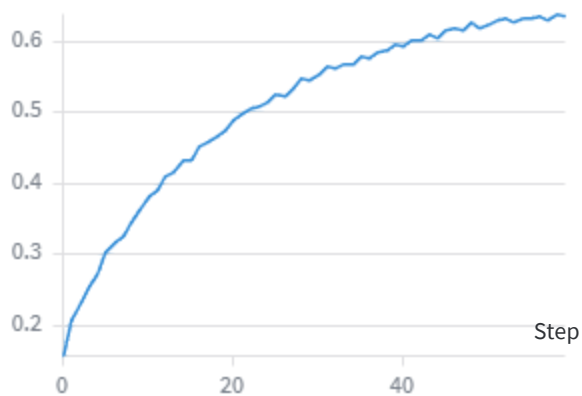
train_precision

— ethereal-river-196



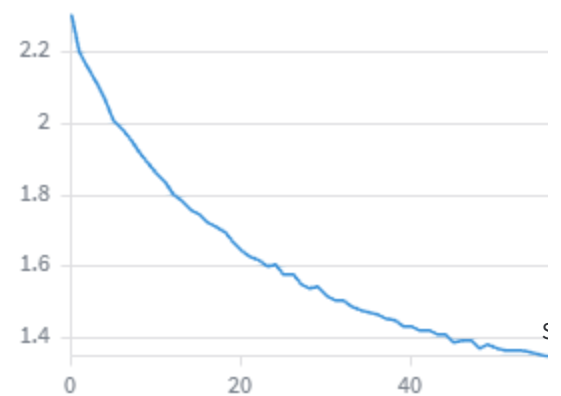
train_recall

— ethereal-river-196

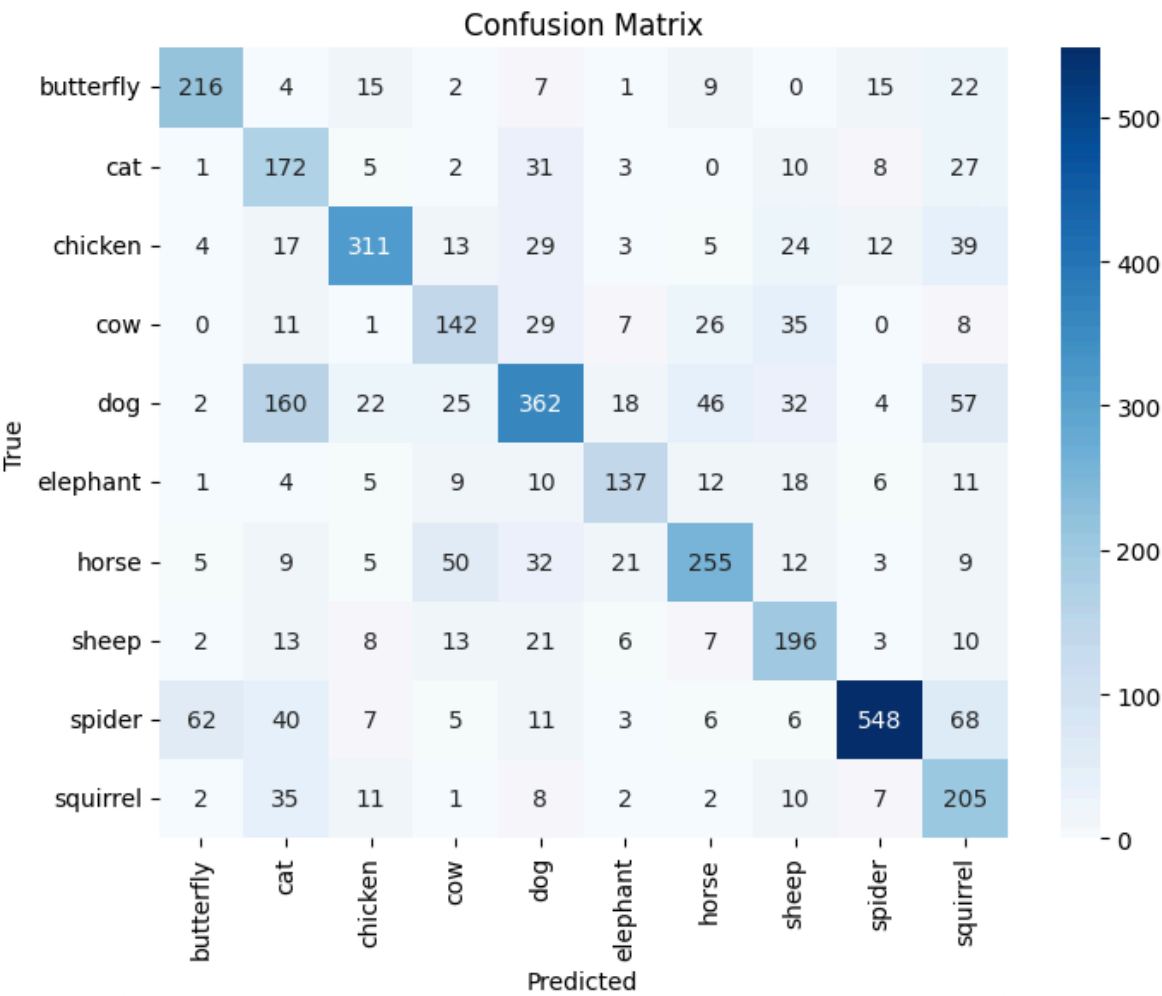


train_loss

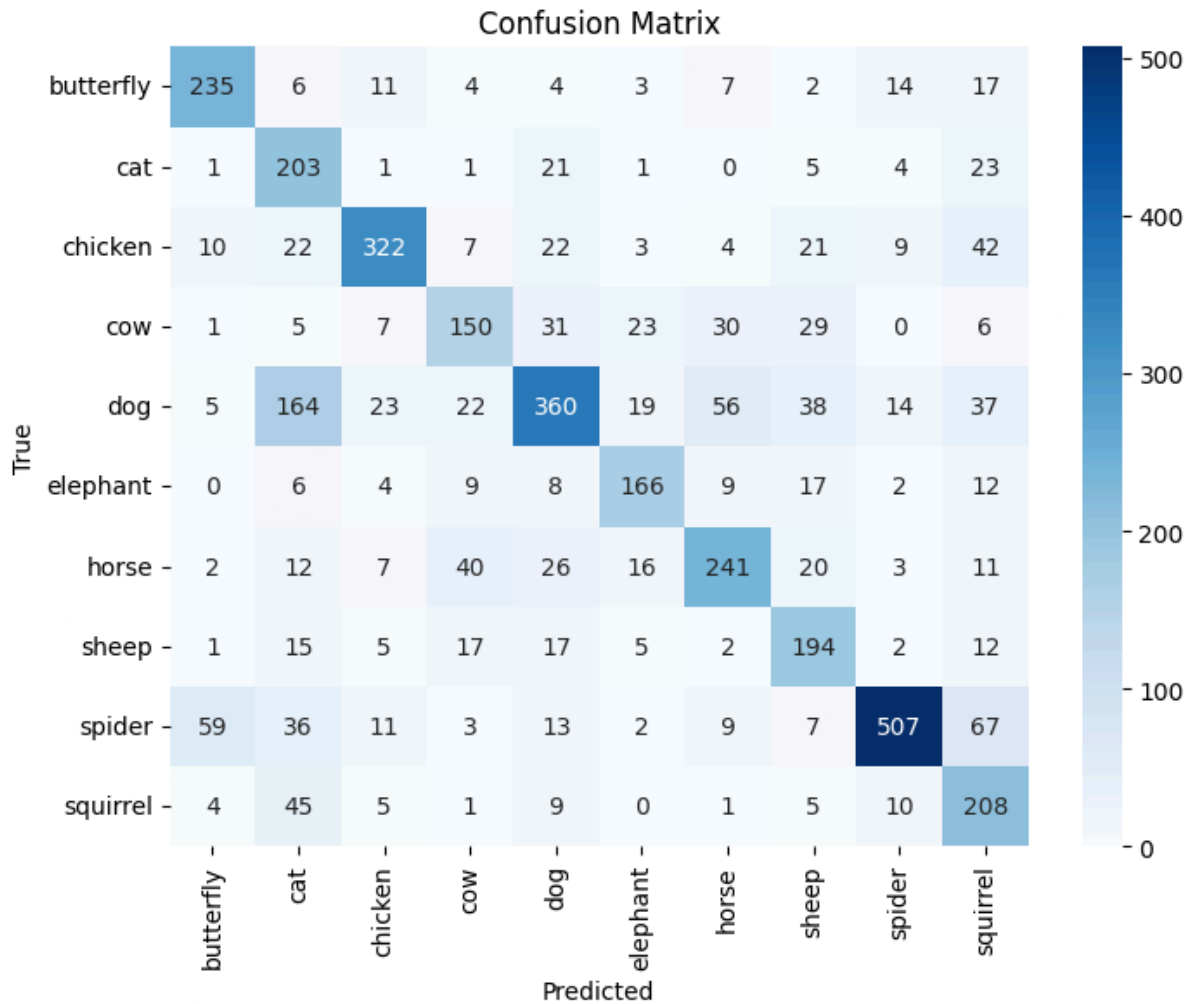
— ethereal-river-196



Dataset	Metric	Value
Train	Accuracy	~63%
	Precision	~63%
	Recall	~63%
	F1-Score	~63%
Validation	Top-1 Accuracy	~63%
	Top-5 Accuracy	~96%
	Accuracy	~63%
	Precision	~68%
	Recall	~63%
	F1-Score	~64%
Test	Accuracy	~66%
	Precision	~65%
	Recall	~67%
	F1-Score	~65%
	Top-1 Accuracy	~65%
	Top-5 Accuracy	~96%



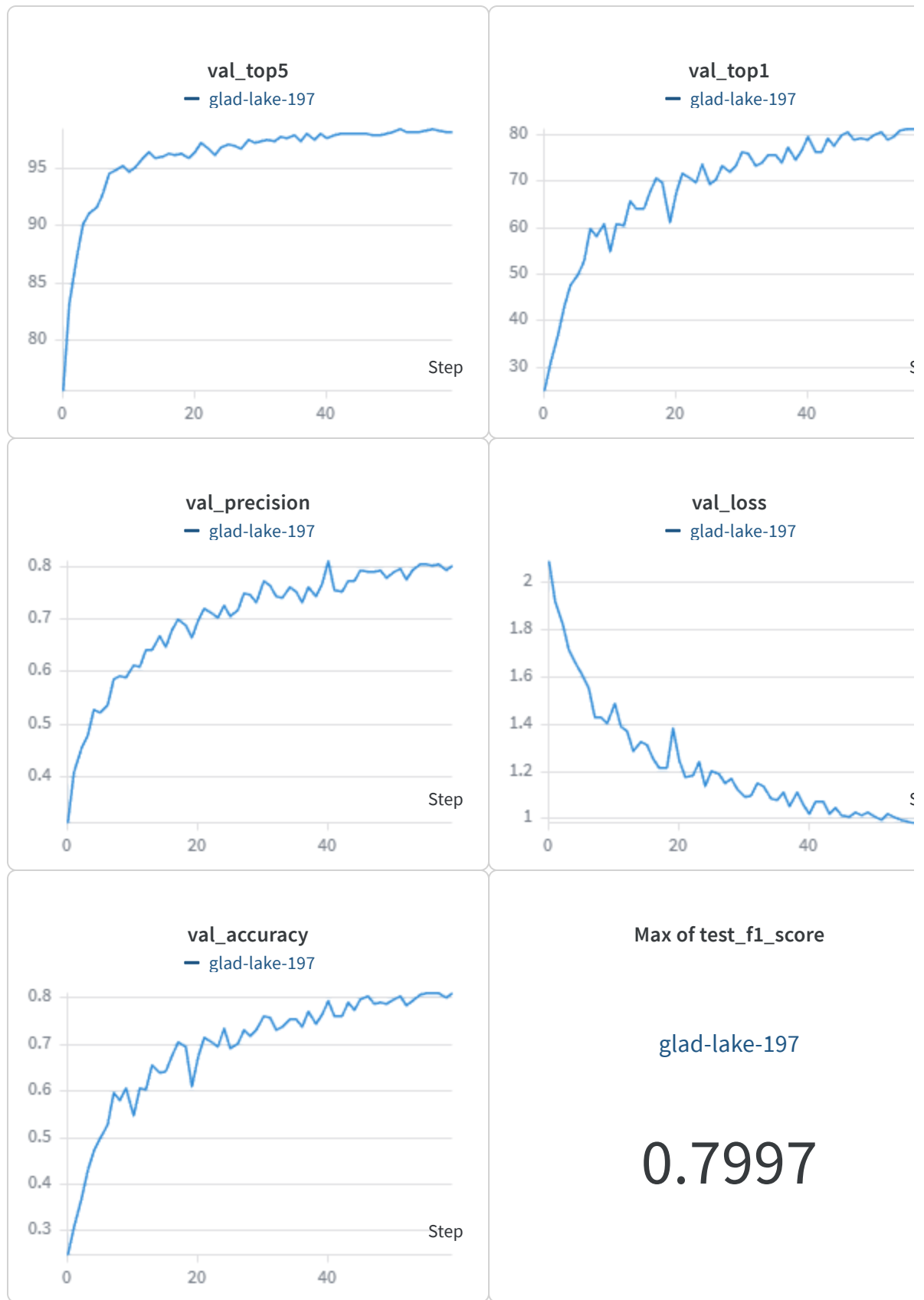
Confusion matrix for the validation set



Confusion matrix for the test set

✓ Oversampled dataset

In this setup, SimpleNetV3 was trained on an oversampled version of the dataset for **60 epochs**, which significantly improved performance across all metrics. Each class has 3397 samples. The model achieved 82% accuracy and precision on the training set, while maintaining strong generalization with 80% accuracy, precision, recall, and F1-score on the validation set. Additionally, a top-5 accuracy of 98% on the validation set shows the model is highly confident and often includes the correct label among its top predictions. Oversampling clearly helped the model learn more robust representations of underrepresented classes.



Max of test_accuracy

glad-lake-197

0.8096

Max of test_top1

glad-lake-197

80.939

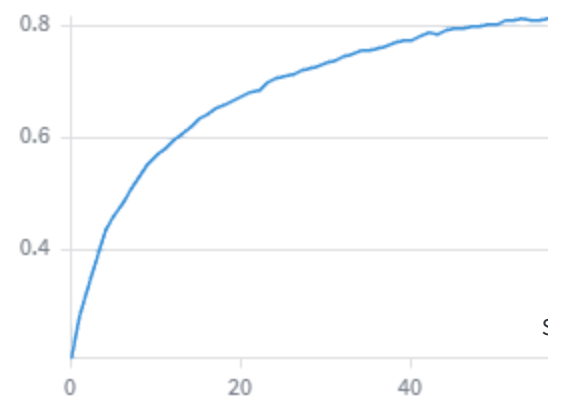
Max of test_recall

glad-lake-197

0.8019

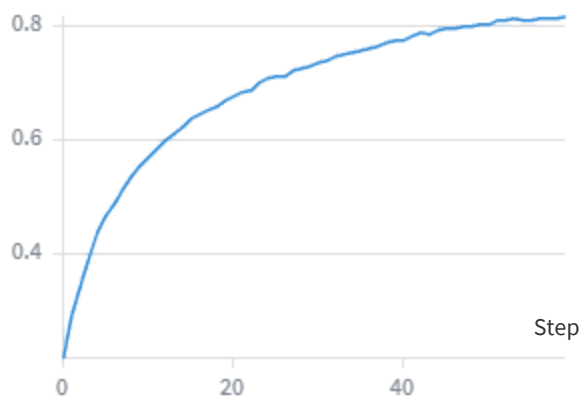
train_f1

— glad-lake-197



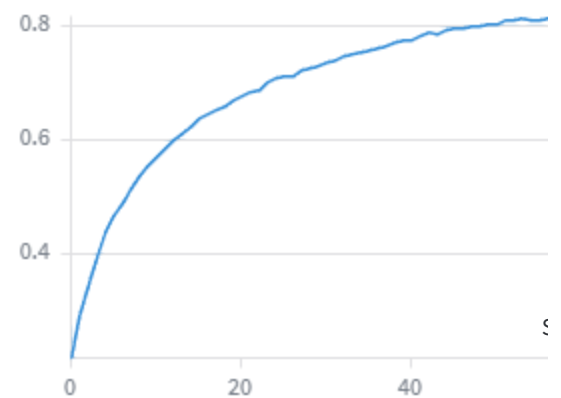
train_accuracy

— glad-lake-197

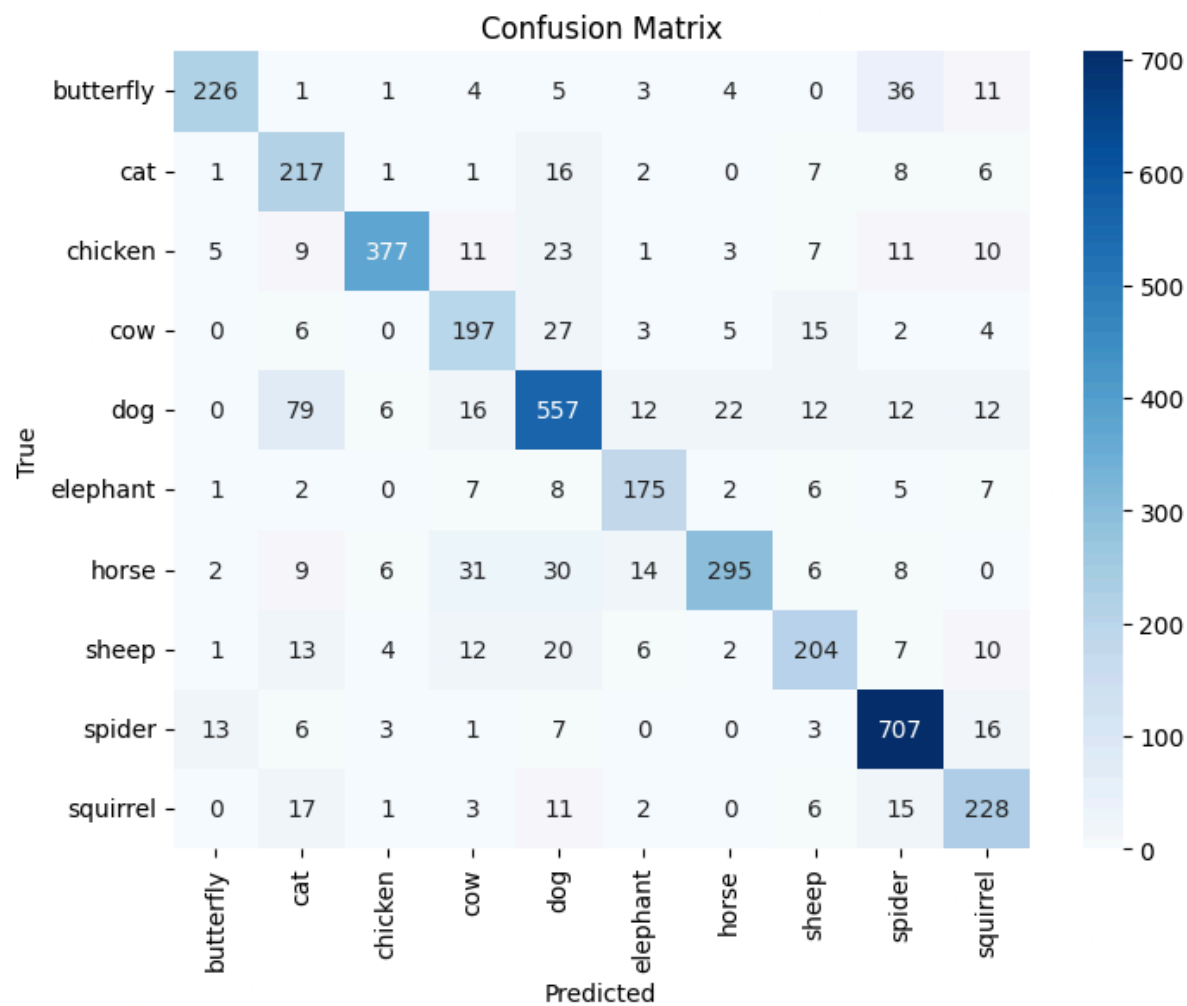


train_recall

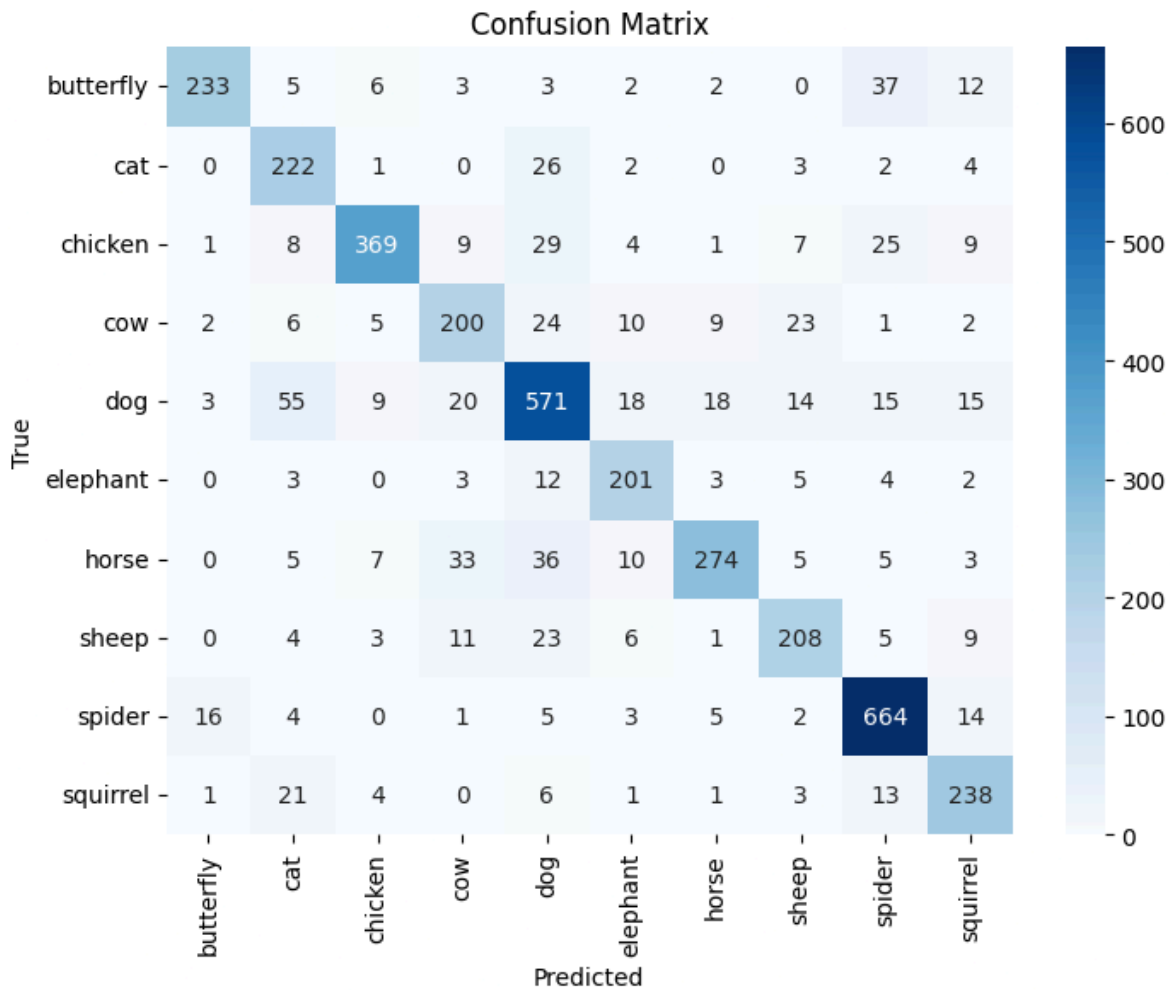
— glad-lake-197



Dataset	Metric	Value
Train	Accuracy	~82%
	Precision	~82%
	Recall	~82%
	F1-Score	~82%
Validation	Top-1 Accuracy	~81%
	Top-5 Accuracy	~98%
	Accuracy	~81%
	Precision	~80%
	Recall	~80%
	F1-Score	~80%
Test	Accuracy	~81%
	Precision	~80%
	Recall	~80%
	F1-Score	~80%
	Top-1 Accuracy	~81%
	Top-5 Accuracy	~98%



Confusion matrix for the validation set



Confusion matrix for the test set

✓ Original imbalanced dataset

Number of samples for each class:

Horse: 1844

Sheep: 1271

Elephant: 1000

Cow: 1325

Butterfly: 1518

Chicken: 2179

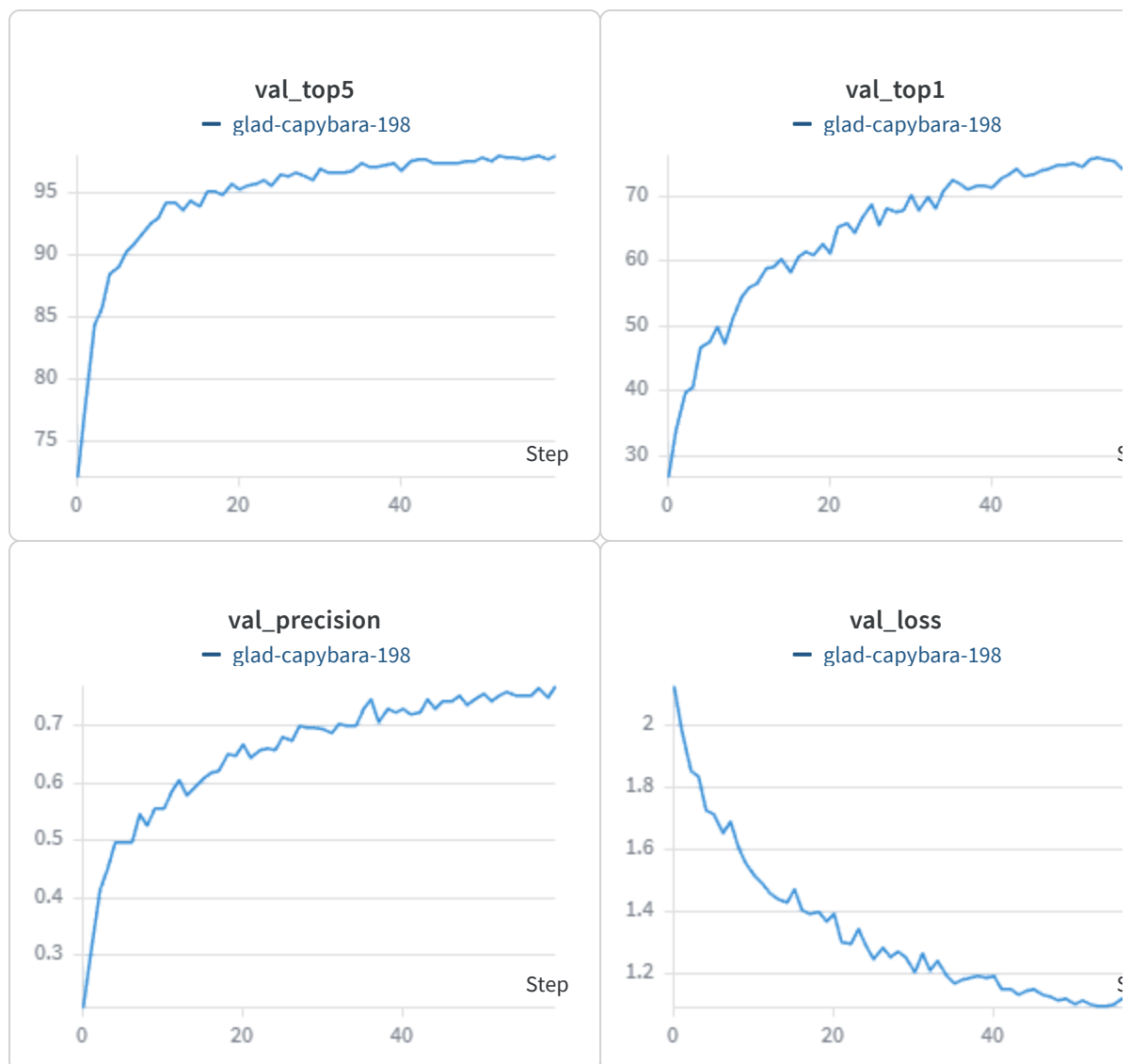
Spider: 3351

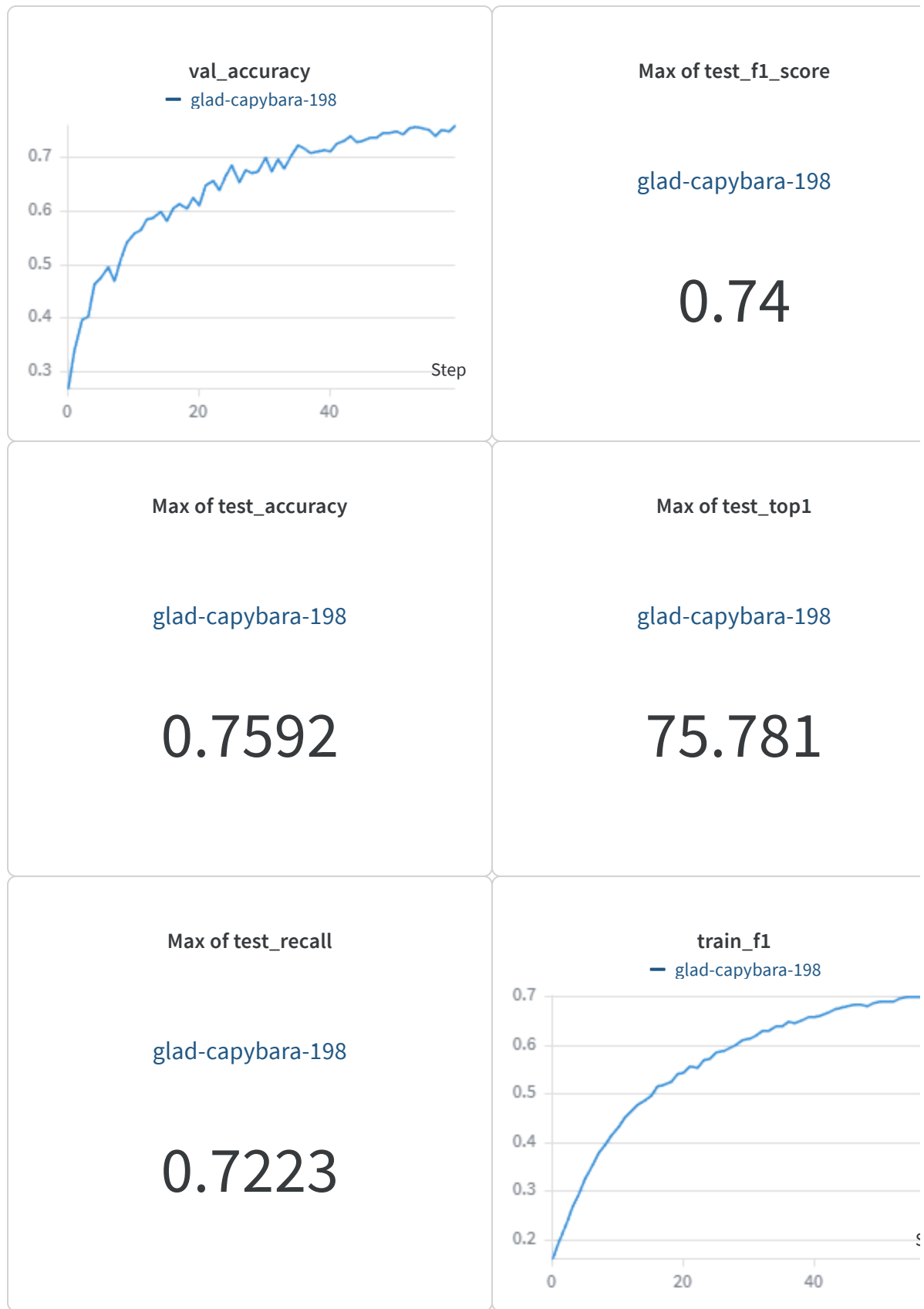
Cat: 1149

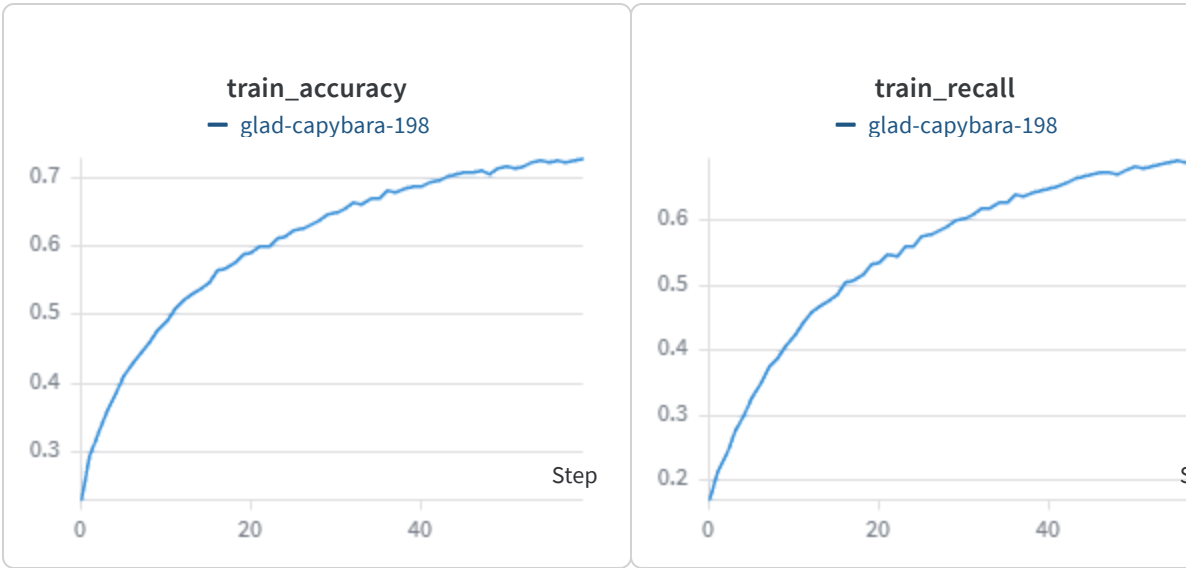
Dog: 3397

Squirrel: 1291

In this run, we trained SimpleNetV3 on the original imbalanced dataset using the same architecture and hyperparameters as in previous experiments. Despite the class imbalance, the model performed surprisingly well, achieving 73% accuracy on the training set and 75% on validation. The top-5 accuracy of 98% remained consistent with the oversampled version, showing that the model is still confidently predicting. However, compared to the balanced training strategies (undersampling and oversampling), the imbalanced setup shows that class distribution does impact overall performance.

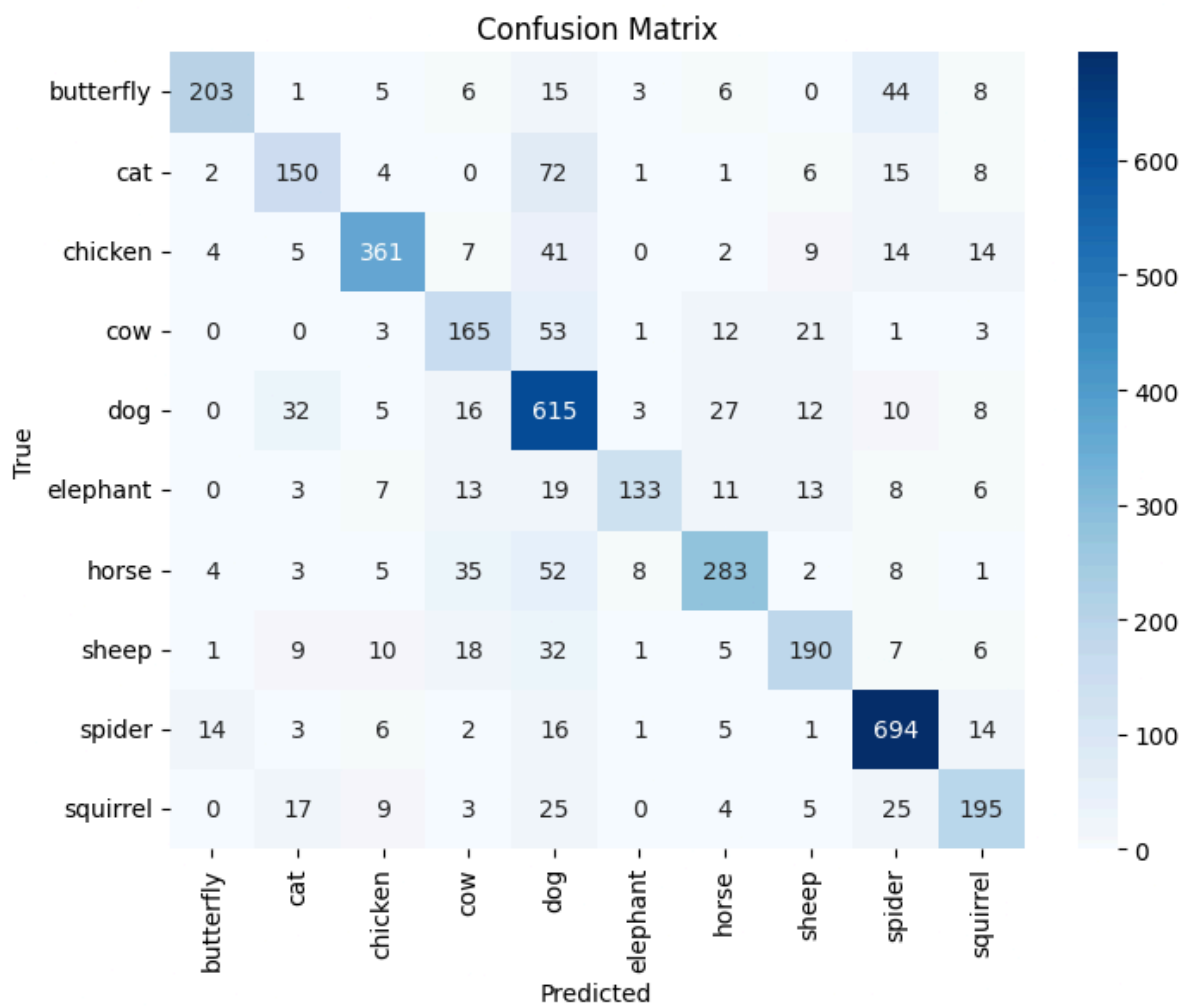




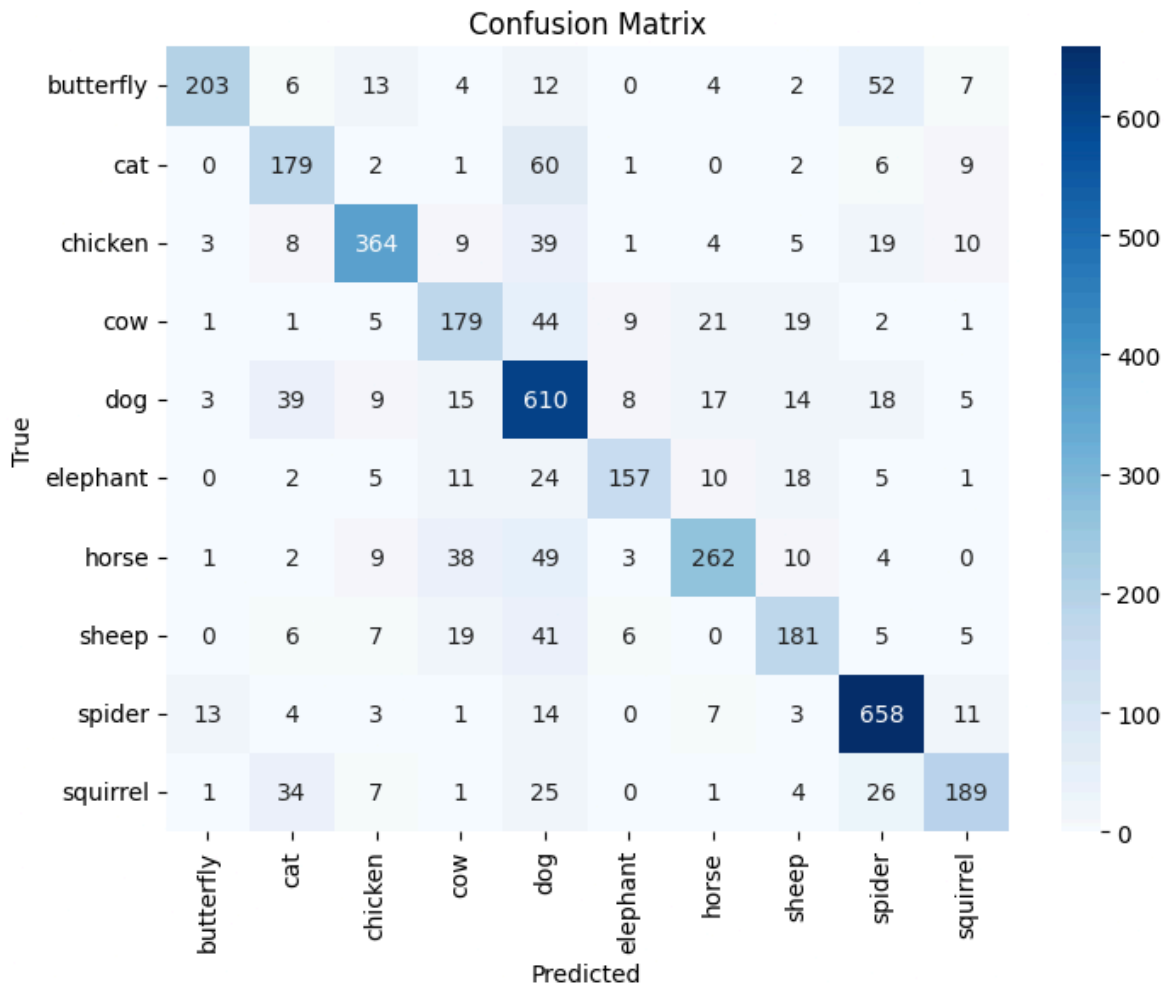


Dataset	Metric	Value
Train	Accuracy	~73%
	Precision	~72%
	Recall	~70%
	F1-Score	~70%
Validation	Top-1 Accuracy	~76%
	Top-5 Accuracy	~98%
	Accuracy	~76%
	Precision	~77%
	Recall	~72%
	F1-Score	~74%

Test	Accuracy	~76%
	Precision	~77%
	Recall	~72%
	F1-Score	~74%
	Top-1 Accuracy	~76%
	Top-5 Accuracy	~98%



Confusion matrix for the validation set



Confusion matrix for the test set

✓ Comparison with paper

Our project confirmed that imbalance within the dataset led to significant variations in performance metrics, aligning with the findings presented in the paper. However, our results indicate that the impact of this imbalance is primarily dependent on the number of samples per class rather than the total number of samples.

We will conduct a thorough comparison of our evaluations with those outlined in the paper:

- In our project, the oversampling method emerged as the most effective approach, producing positive outcomes across all metrics we analyzed.
- We concur with the assertion that "if training time is a concern, undersampling is a preferable choice, as it significantly reduces the size

